



AGENDA

- C# Programming
- Module 1 – Introductions - What is C#
- Q&A

C# Tutorial Summary

C# (C Sharp) is one of the most popular programming languages which is widely used for building Windows applications, mobile applications, and games. This course is taught in a practical GOAL oriented way to learn C# programming. It is recommended for you to practice the code assignments given after each C Sharp tutorial to learn C Sharp fast and easily.

What is C#?

C# (C Sharp) is a general purpose, multi-paradigm programming language developed by Microsoft that runs on the .NET Framework. C# is widely used for building mobile applications, games and windows applications.

Thank you



AGENDA

- C# Programming
- Module 2 – High-level Introduction to C#
- Q&A

What is .Net Framework?

.Net Framework is a software development platform developed by Microsoft for building and running Windows applications. The .Net framework consists of developer tools, programming languages, and libraries to build desktop and web applications. It is also used to build websites, web services, and games.

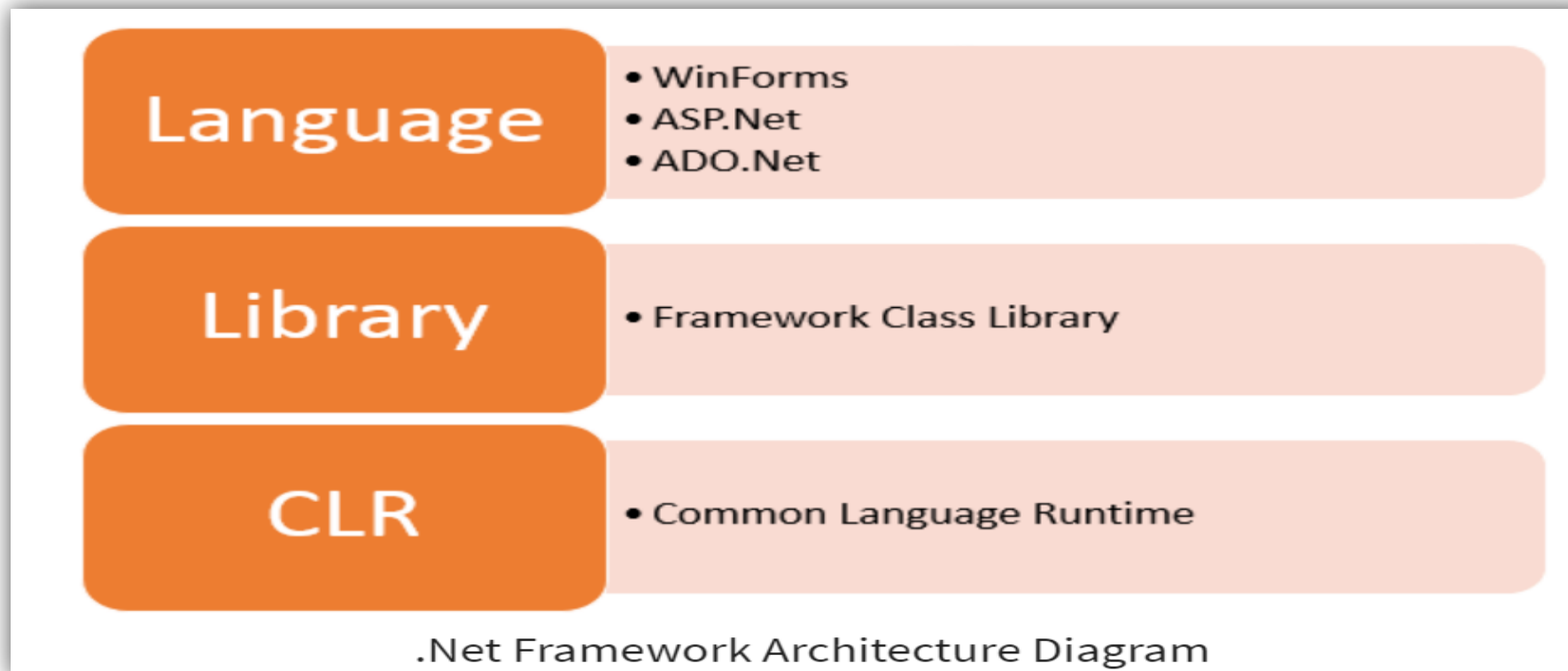
The .Net framework was meant to create applications, which would run on the Windows Platform. The first version of the .Net framework was released in the year 2002. The version was called .Net framework 1.0. The Microsoft .Net framework has come a long way since then, and the current version is .Net Framework 4.7.2.

The Microsoft .Net framework can be used to create both – **Form-based** and **Web-based** applications. [Web services](#) can also be developed using the .Net framework. The framework also supports various programming languages such as Visual Basic and C#. So developers can choose and select the language to develop the required application. In this chapter, you will learn some basics of the .Net framework.

.Net Framework Architecture

.Net Framework Architecture is a programming model for the .Net platform that provides an execution environment and integration with various programming languages for simple development and deployment of various Windows and desktop applications. It consists of class libraries and reusable components.

The basic architecture of the .Net framework is as shown below.



. NET Components

The architecture of .Net framework is based on the following key components;

1. Common Language Runtime

The “Common Language Infrastructure” or CLI is a platform in .Net architecture on which the .Net programs are executed.

The CLI has the following key features:

Exception Handling – Exceptions are errors which occur when the application is executed.

Examples of exceptions are:

- If an application tries to open a file on the local machine, but the file is not present.
- If the application tries to fetch some records from a [database](#), but the connection to the database is not valid.

Garbage Collection – Garbage collection is the process of removing unwanted resources when they are no longer required.

Examples of garbage collection are

- A File handle which is no longer required. If the application has finished all operations on a file, then the file handle may no longer be required.
- The database connection is no longer required. If the application has finished all operations on a database, then the database connection may no longer be required.

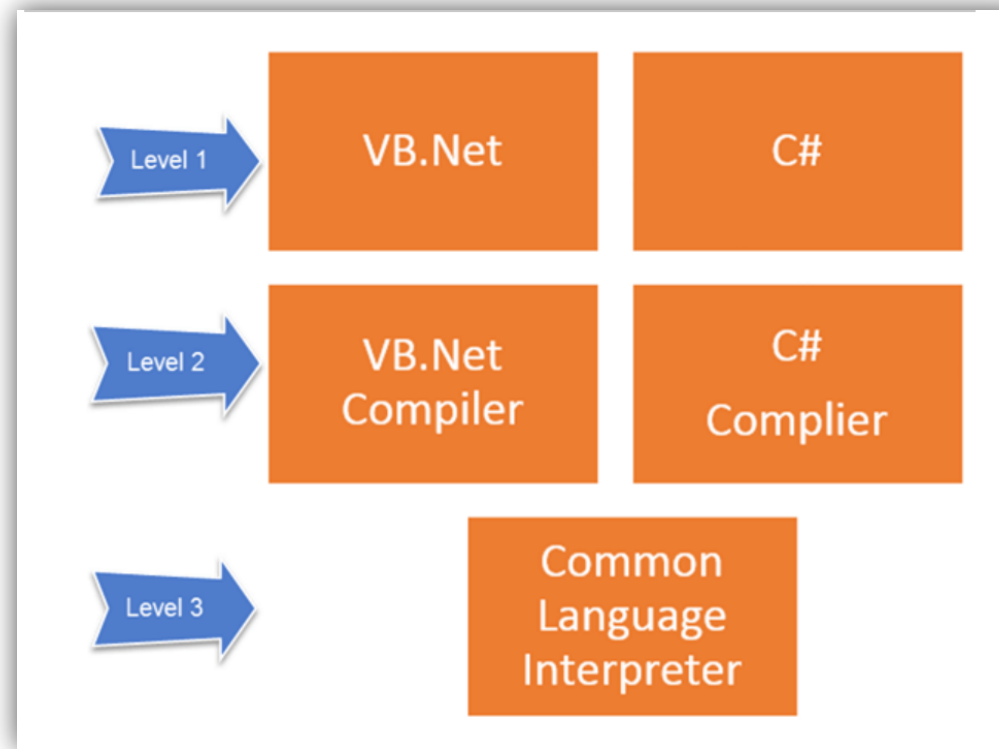
Working with Various programming languages –

As noted in an earlier section, a developer can develop an application in a variety of .Net programming languages.

1.Language – The first level is the programming language itself, the most common ones are VB.Net and C#.

2.Compiler – There is a compiler which will be separate for each programming language. So underlying the VB.Net language, there will be a separate VB.Net compiler. Similarly, for C#, you will have another compiler.

3.Common Language Interpreter – This is the final layer in .Net which would be used to run a .net program developed in any [programming language](#). So the subsequent compiler will send the program to the CLI layer to run the .Net application.



2. Class Library

The .NET Framework includes a set of standard class libraries. A class library is a collection of methods and functions that can be used for the core purpose.

For example, there is a class library with methods to handle all file-level operations. So there is a method which can be used to read the text from a file. Similarly, there is a method to write text to a file.

Most of the methods are split into either the System.* or Microsoft.* namespaces. (The asterisk * just means a reference to all of the methods that fall under the System or Microsoft namespace)

A namespace is a logical separation of methods. We will learn these namespaces more in detail in the subsequent chapters.

3. Languages

The types of applications that can be built in the .Net framework is classified broadly into the following categories.

WinForms – This is used for developing Forms-based applications, which would run on an end user machine. Notepad is an example of a client-based application.

ASP.Net – This is used for developing web-based applications, which are made to run on any browser such as Internet Explorer, Chrome or Firefox.

- The Web application would be processed on a server, which would have Internet Information Services Installed.
- Internet Information Services or IIS is a Microsoft component which is used to execute an [Asp.Net](#) application.
- The result of the execution is then sent to the client machines, and the output is shown in the browser.

ADO.Net – This technology is used to develop applications to interact with Databases such as Oracle or Microsoft [SQL](#) Server. Microsoft always ensures that .Net frameworks are in compliance with all the supported Windows operating systems.

.Net Framework Design Principle

Now in this .Net Architecture tutorial, we will learn the design principles of .Net framework. The following design principles of the .Net framework is what makes it very relevant to create .Net based applications.

- 1) Interoperability** – The .Net framework provides a lot of backward support. Suppose if you had an application built on an [older version of the .Net framework](#), say 2.0. And if you tried to run the same application on a machine which had the higher version of the .Net framework, say 3.5. The application would still work. This is because with every release, Microsoft ensures that older framework versions gel well with the latest version.
- 2) Portability** – Applications built on the .Net framework can be made to work on any Windows platform. And now in recent times, Microsoft is also envisioning to make Microsoft products work on other platforms, such as iOS and [Linux](#).
- 3) Security** – The .NET Framework has a good security mechanism. The inbuilt security mechanism helps in both validation and verification of applications. Every application can explicitly define their security mechanism. Each security mechanism is used to grant the user access to the code or to the running program.
- 4) Memory management** – The Common Language runtime does all the work or [memory management](#). The .Net framework has all the capability to see those resources, which are not used by a running program. It would then release those resources accordingly. This is done via a program called the “Garbage Collector” which runs as part of the .Net framework. The garbage collector runs at regular intervals and keeps on checking which system resources are not utilized, and frees them accordingly.
- 5) Simplified deployment** – The .Net framework also have tools, which can be used to package applications built on the .Net framework. These packages can then be distributed to client machines. The packages would then automatically install the application.

.Net Framework Version History

The first version of the .Net framework was released in the year 2002. The version was called .Net framework 1.0. The .Net framework has come a long way since then, and the current version is 4.7.1.

Below is the table of .Net framework versions, which have been released with their release dates. Every version has relevant changes to the framework.

For example, in framework 3.5 and onwards a key framework called the **Entity framework** was released. This framework is used to change the approach in which the applications are developed while working with [databases](#).

Version number	CLR version	Release date
1.0	1.0	2002-02-13
1.1	1.1	2003-04-24
2.0	2.0	2005-11-07
3.0	2.0	2006-11-06
3.5	2.0	2007-11-19
4.0	4	2010-04-12
4.5	4	2012-08-15
4.5.1	4	2013-10-17
4.5.2	4	2014-05-05
4.6	4	2015-07-20
4.6.1	4	2015-11-17
4.6.2	4	2016-08-02
4.7	4	2017-04-05
4.7.1	4	2017-10-17

The biggest advantage of the [.Net framework](#) is that it supports Windows platform. Almost everyone works with Windows machines. Microsoft always ensures that .Net frameworks are in compliance with all the supported Windows operating systems.

C# Version History

Version	.NET Framework	Visual Studio	Important Features
C# 1.0	.NET Framework 1.0/1.1	Visual Studio .NET 2002	First release of C#
C# 2.0	.NET Framework 2.0	Visual Studio 2005	- Generics - Partial types - Anonymous methods - Nullable types - Iterators - Covariance and contravariance
C# 3.0	.NET Framework 3.0/3.5	Visual Studio 2008	- Auto-implemented properties - Anonymous types - Query expressions - Lambda expression - Expression trees - Extension methods
C# 4.0	.NET Framework 4.0	Visual Studio 2010	- Dynamic binding - Named/optional arguments - Generic covariant and contravariant - Embedded interop types
C# 5.0	.NET Framework 4.5	Visual Studio 2012/2013	- Asynchronous members - Caller info attributes

C# 6.0	.NET Framework 4.6	Visual Studio 2013/2015	- Static imports - Exception filters - Property initializers - Expression bodied members - Null propagator - String interpolation - nameof operator - Dictionary initializer
--------	--------------------	-------------------------	--

C# 7.0	.NET Core	Visual Studio 2017	- Improved performance and productivity - Azure Support - AI Support - Game development - Cross platform - Mobile App Development - Window App Development
--------	-----------	--------------------	--

How to Download and Install Visual Studio for C# in Windows

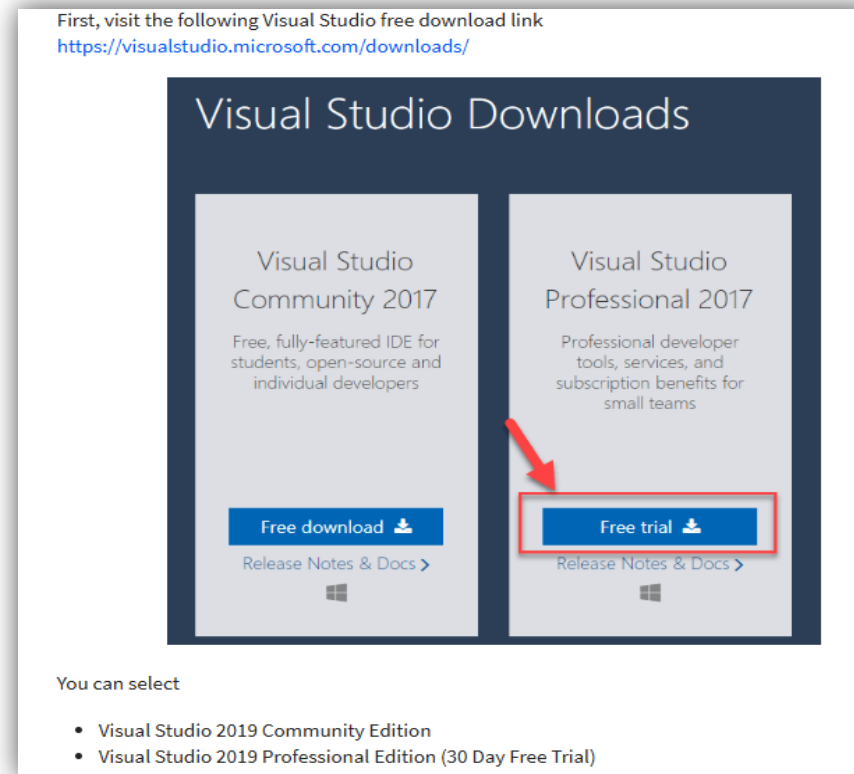
Microsoft Visual Studio is an integrated development environment (IDE) from Microsoft. It is used to develop computer programs for Microsoft Windows. Visual Studio is one stop shop for all applications built on the .Net platform. One can develop, debug and run applications using Visual Studio.

Both Forms-based and web-based applications can be designed and developed using this IDE. Below is a step by step process on how to install Visual Studio.

How to Download and Install Visual Studio for Windows

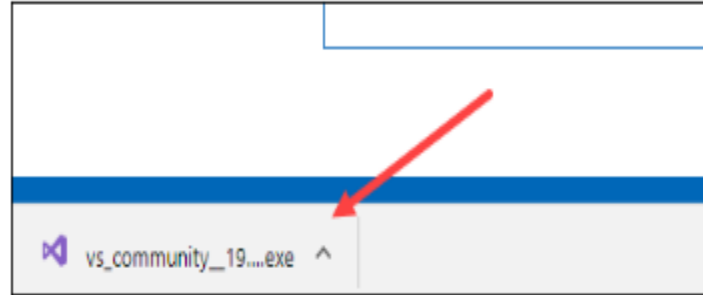
Let's look at the Visual Studio installation steps and download Visual Studio IDE so that we can work with creating programs in the subsequent tutorials. In this tutorial, we will learn how to download Visual Studio for Windows.

Step 1) Download Visual Studio



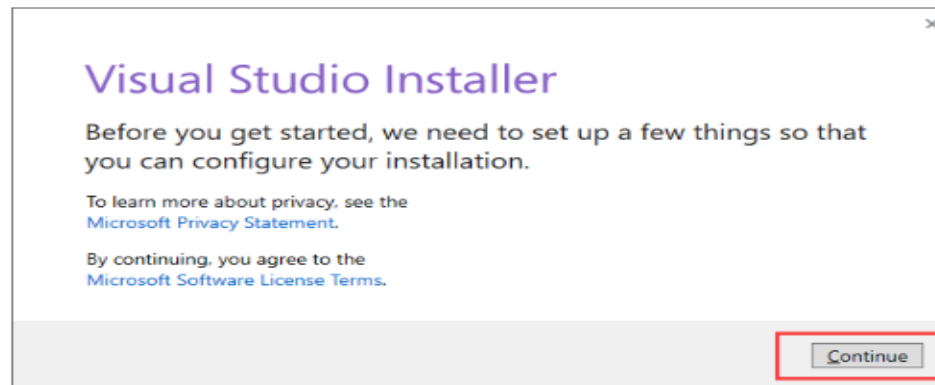
Step 2) Open the .exe file

Click on the downloaded exe file

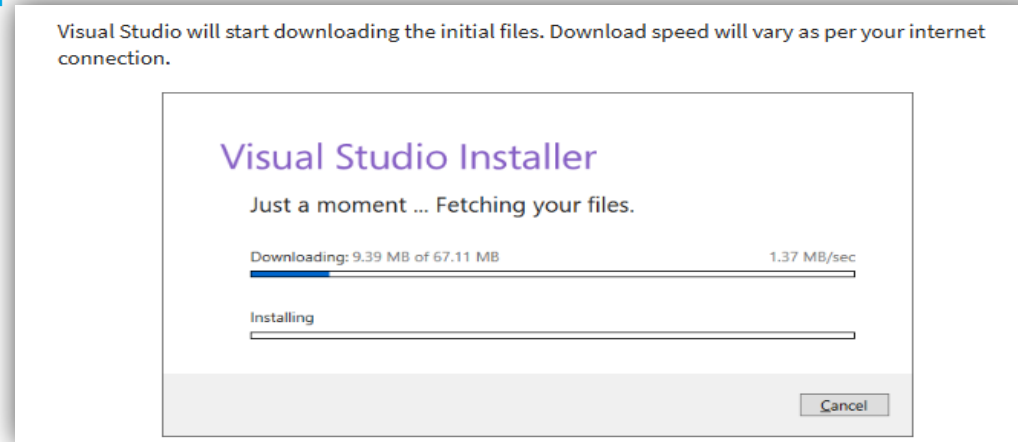


Step 3) Start the installation

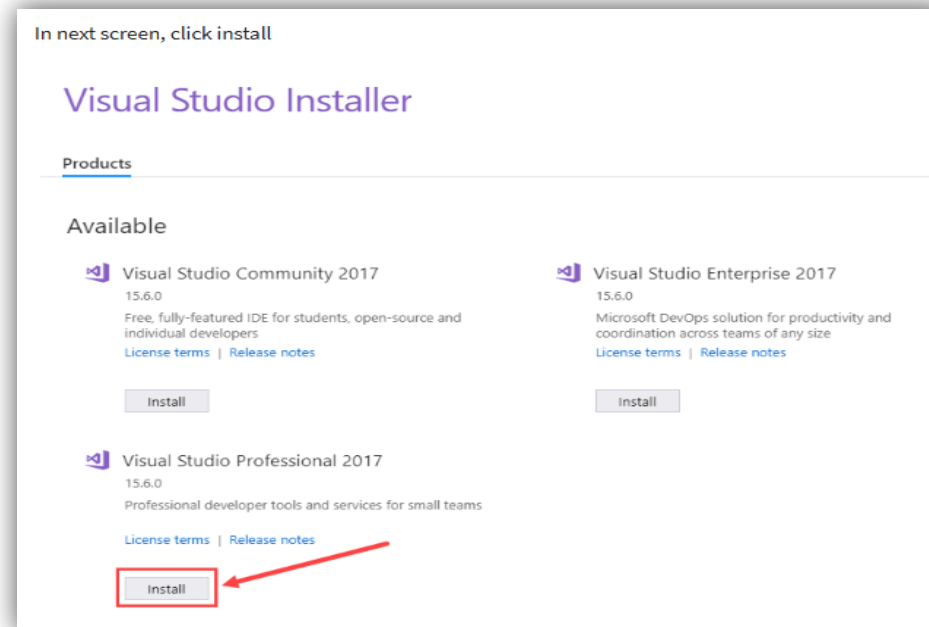
In the next screen, click continue to start Visual Studio installation



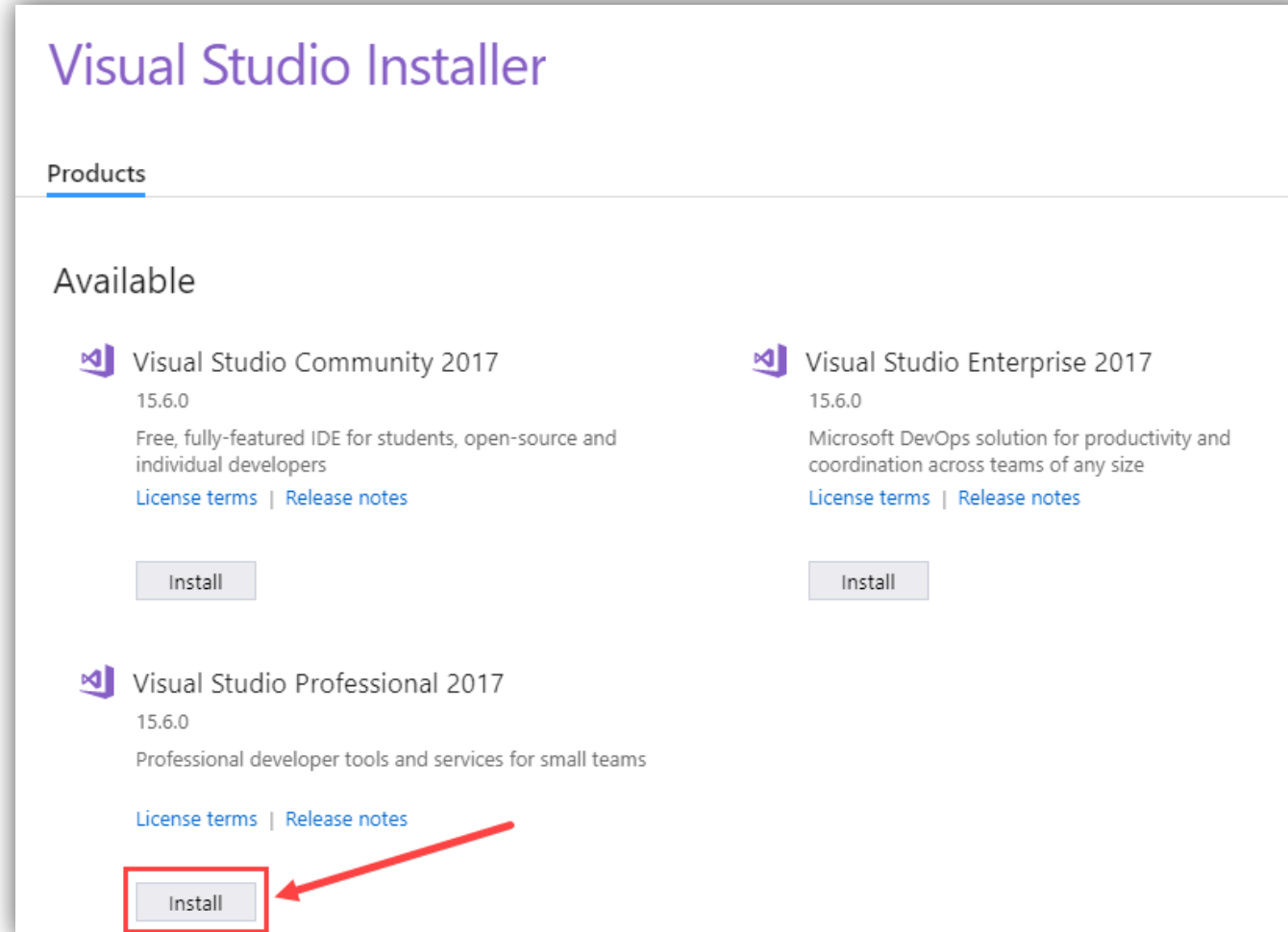
Step 4) Let the installation complete



Step 5) Choose the software version

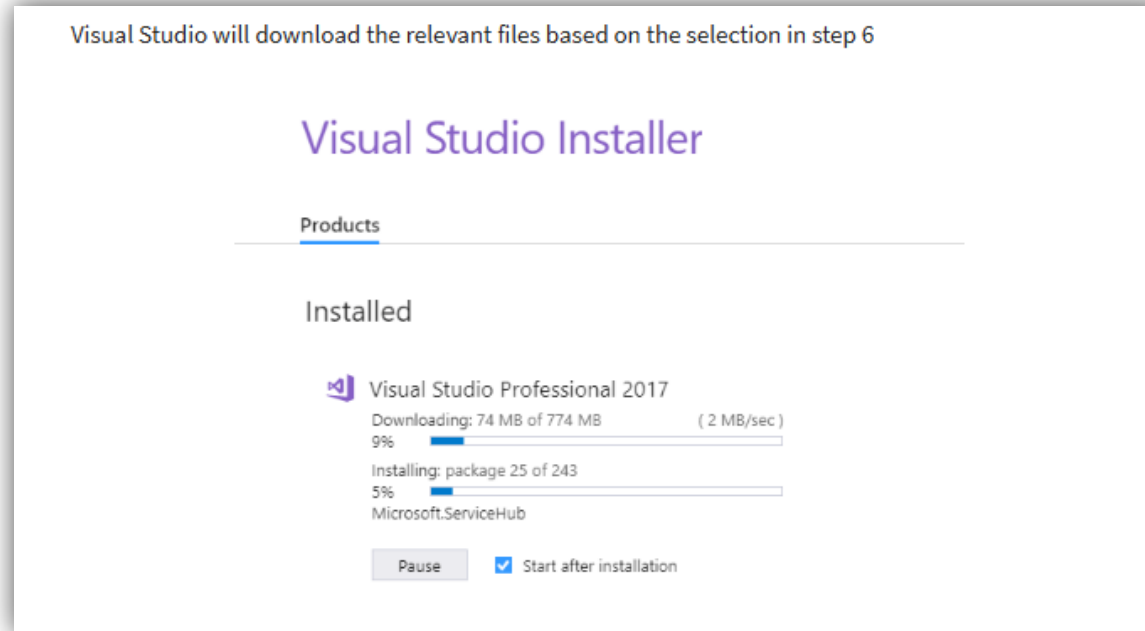


Step 6) Select the desktop

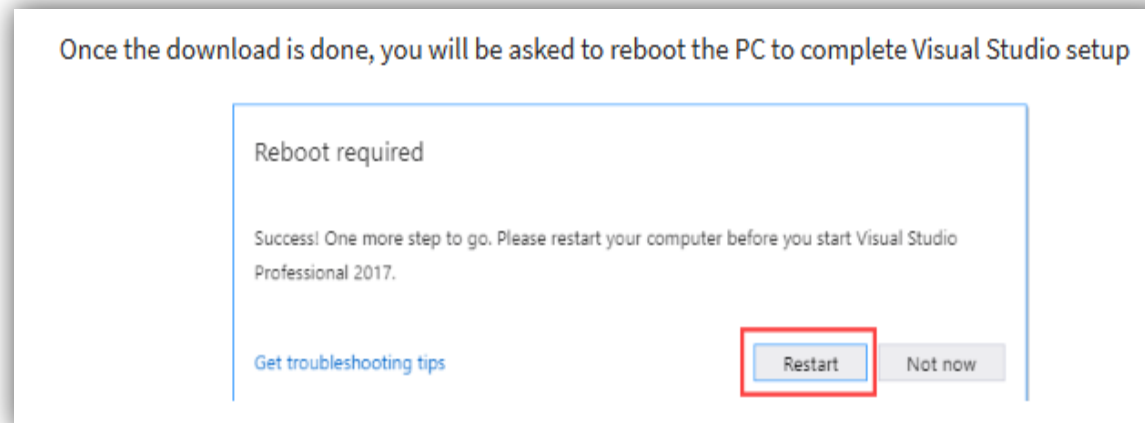


1. Select ".Net desktop development"
2. Click install

Step 7) Wait for the files to be downloaded

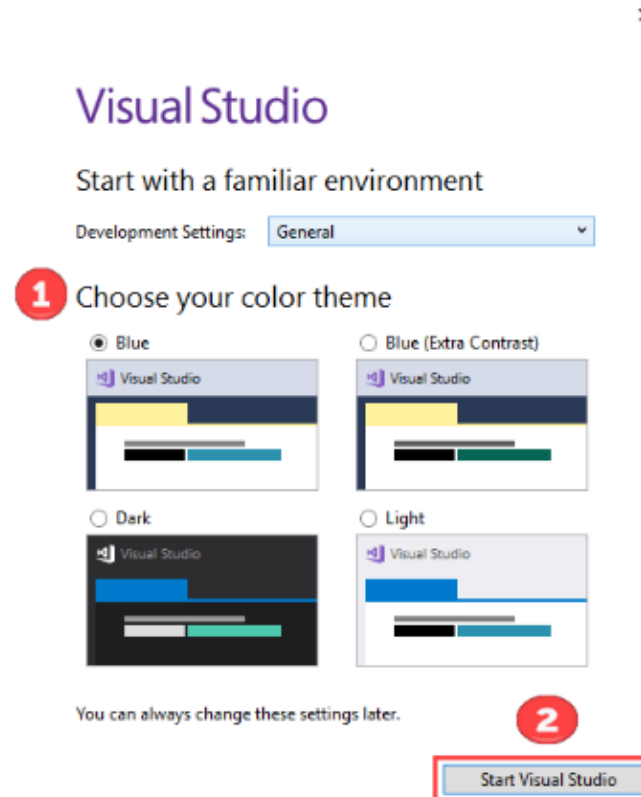


Step 8) Reboot your pc



Step 9) Open Visual Studio

Post reboot, open the Visual Studio IDE

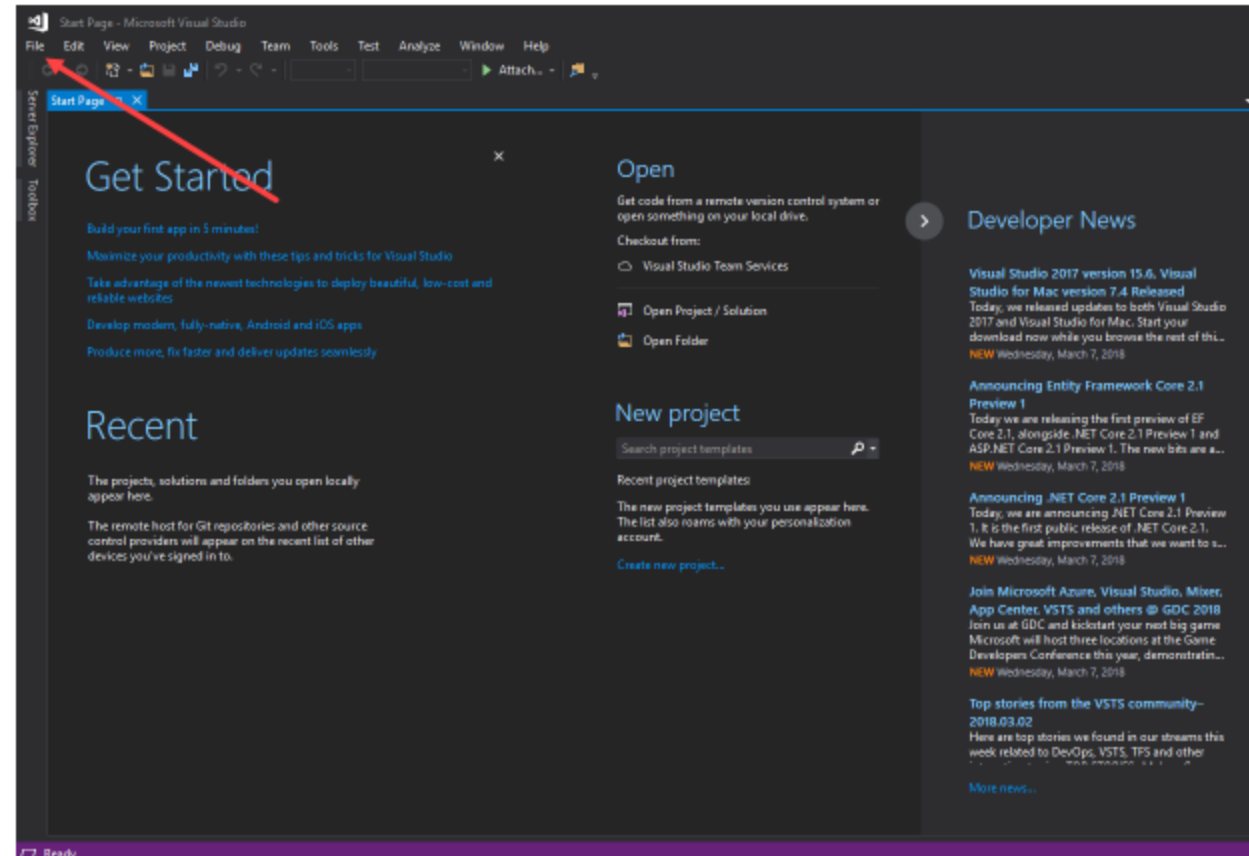


1. Select a theme of your choice
2. Click Start Visual Studio

Step 9) Open Visual Studio

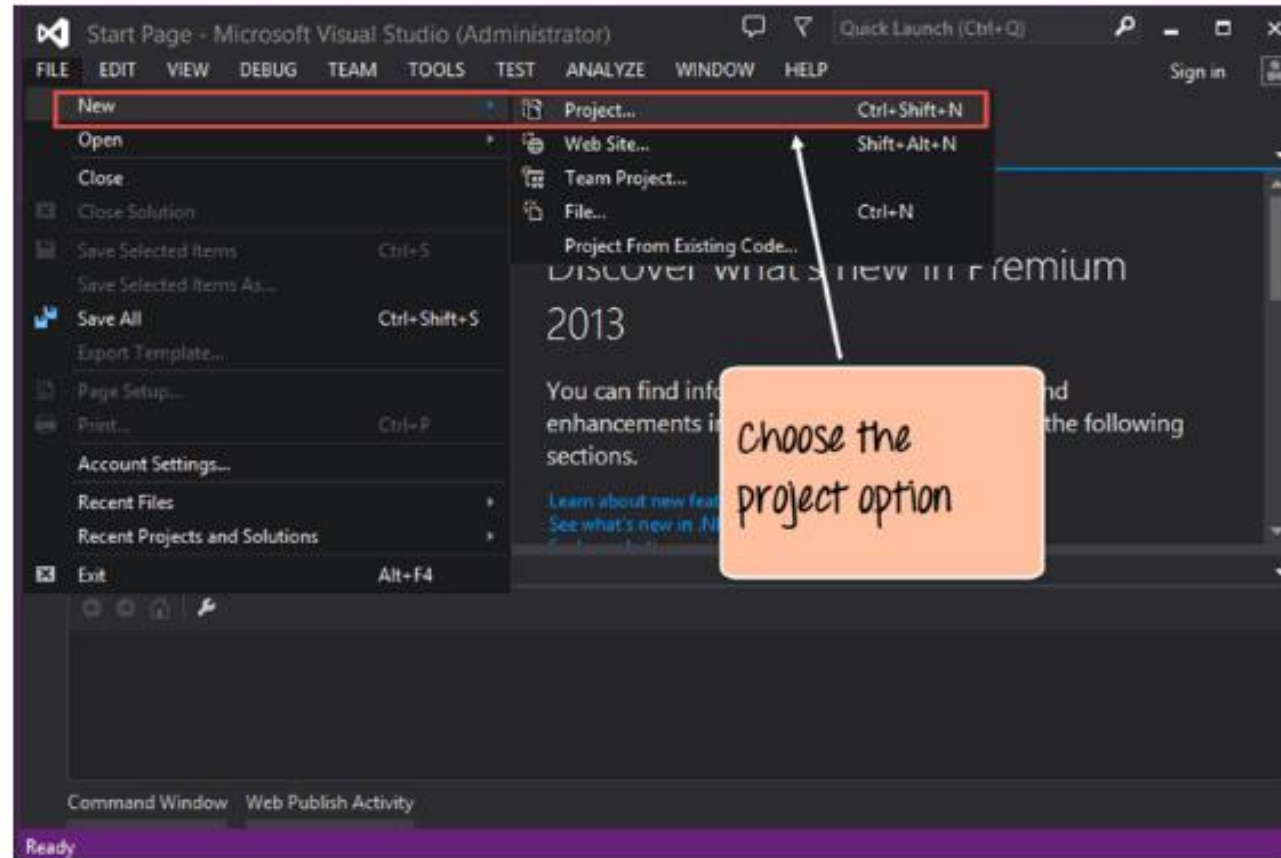
Step 10) Start using Visual Studio

In Visual Studio IDE, you can navigate to File menu to create new C# applications.

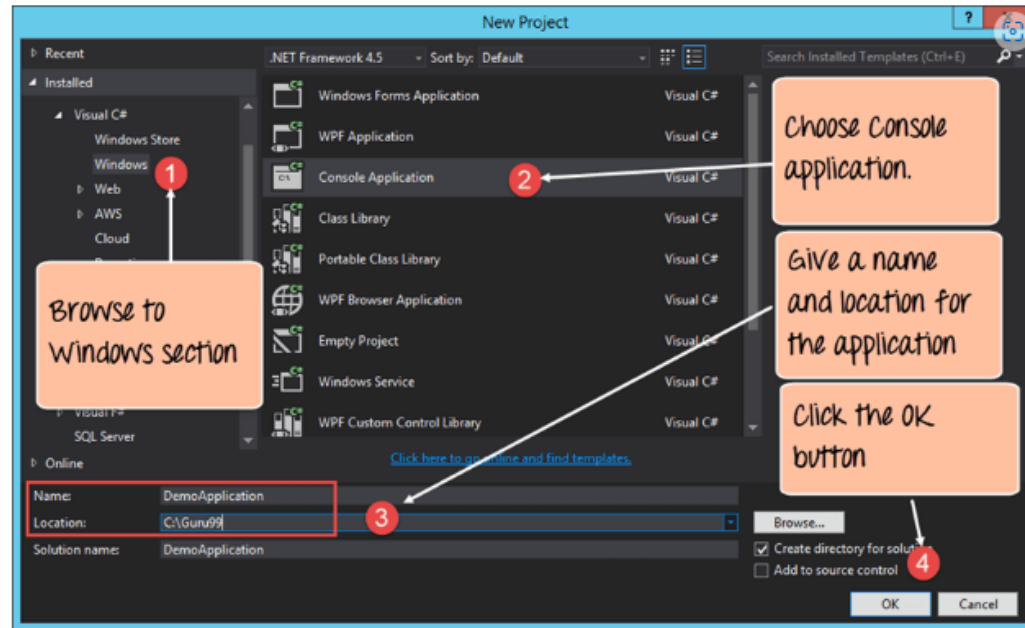


That's it to Visual Studio for C# installation.

Step 1) The first step involves the creation of a new project in Visual Studio. For that, once the Visual Studio is launched, you need to choose the menu option New->Project.



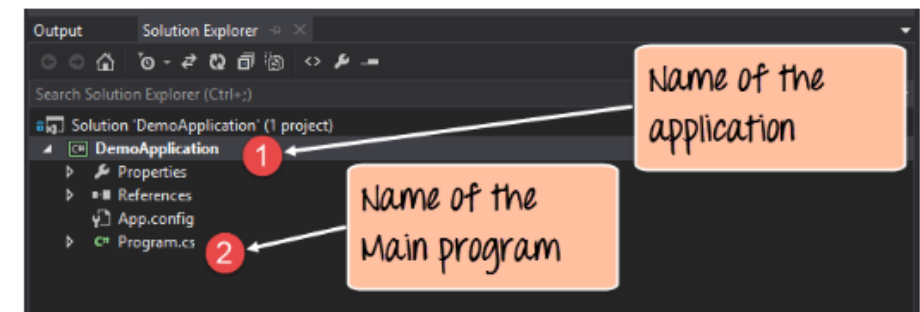
Step 2) The next step is to choose the project type as a Console application. Here, we also need to mention the name and location of our project.



1. In the project dialog box, we can see various options for creating different types of projects in Visual Studio. Click the Windows option on the left-hand side.
2. When we click the Windows options in the previous step, we will be able to see an option for Console Application. Click this option.
3. We then give a name for the application which in our case is DemoApplication. We also need to provide a location to store our application.
4. Finally, we click the 'OK' button to let Visual Studio to create our project.

If the above steps are followed, you will get the below output in Visual Studio.

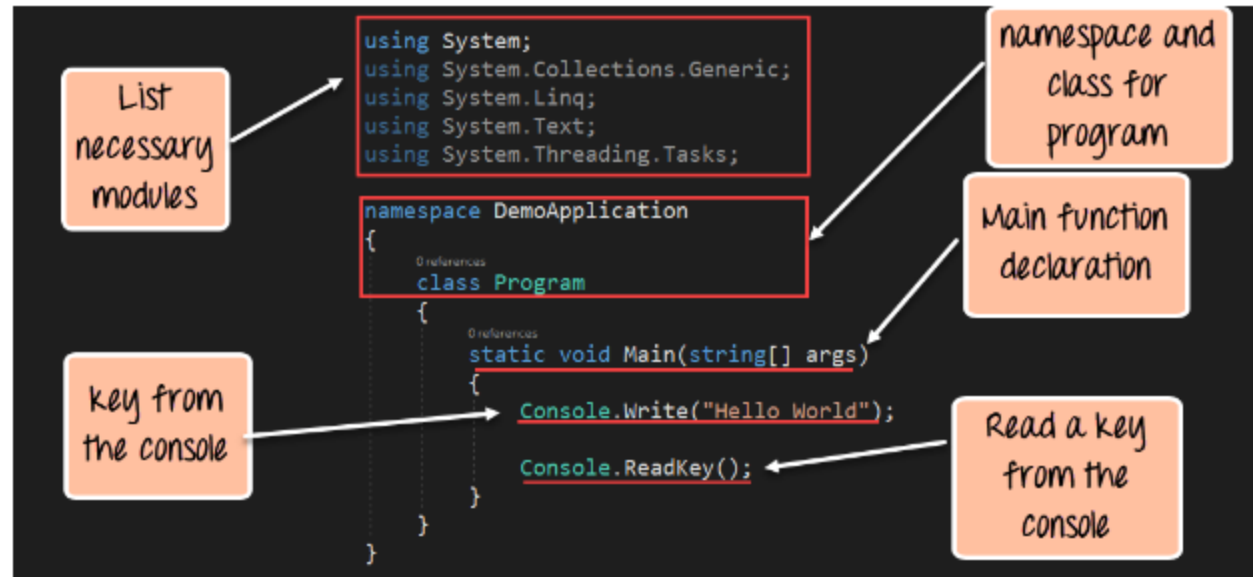
Output:-



1. A project called 'DemoApplication' will be created in Visual Studio. This project will contain all the necessary artifacts required to run the Console application.
2. The Main program called Program.cs is default code file which is created when a new application is created in Visual Studio. This code will contain the necessary code for our console application.

Step 3) Now let's write our code which will be used to display the string "Hello World" in the console application.

All the below code needs to be entered into the Program.cs file. The code will be used to write "Hello World" when the console application runs.

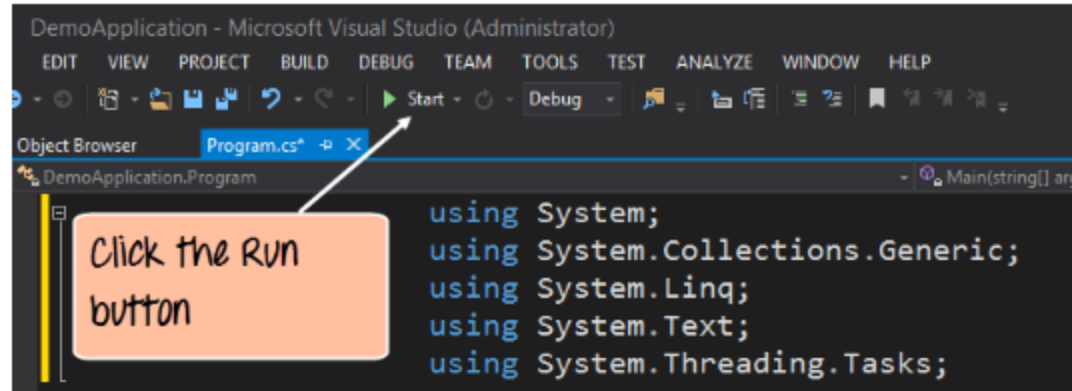


Code Explanation:-

1. The first lines of code are default lines entered by Visual Studio. The 'using' statement is used to import existing .Net modules in our console application. These modules are required for any .Net application to run properly. They contain the bare minimum code to make a code work on a Windows machine.
2. Every application belongs to a class. C# is an object-oriented language, and hence, all code needs to be defined in a self-sustaining module called a 'Class.' In turn, every class belongs to a namespace. A namespace is just a logical grouping of classes.
3. The Main function is a special function which is automatically called when a console application runs. Here you need to ensure to enter the code required to display the required string in the console application.
4. The Console class is available in .Net which allows one to work with console applications. Here we are using an inbuilt method called 'Write' to write the string "Hello World" in the console.
5. We then use the Console.ReadKey() method to read any key from the console. By entering this line of code, the program will wait and not exit immediately. The program will wait for the user to enter any key before finally exiting. If you don't include this statement in code, the program will exit as soon as it is run.

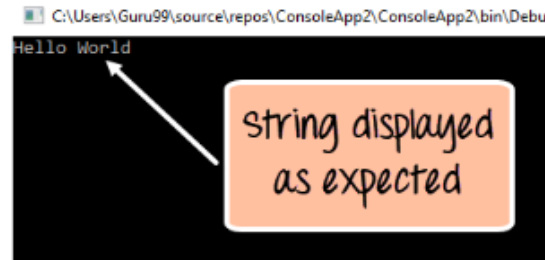
C# Hello World: First Console Application Program

Step 4) Run your [.Net program](#). To run any program, you need to click the Start button in Visual Studio.



If the above code is entered properly and the program is executed successfully, the following output will be displayed.

Output:



From the output, you can clearly see that the string "Hello World" is displayed properly. This is because of the `Console.write` statement causes this string to be sent to the console.

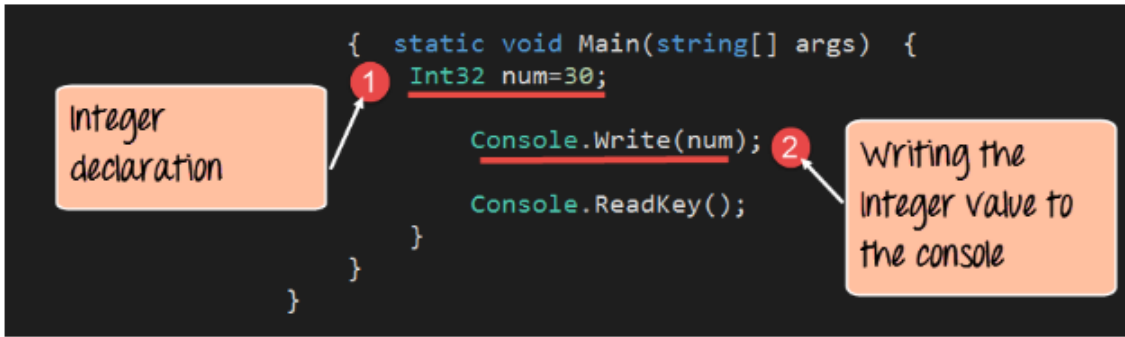
AGENDA

- C# Programming
- Module 3 – C# Data Types
- Q&A



1) Integer

An Integer data types are used to work with numbers. In this case, the numbers are whole numbers like 10, 20 or 30. In C#, the datatype is denoted by the **Int32 keyword**. Below is an example of how this datatype can be used. In our example, we will define an Int32 variable called num. We will then assign an Integer value to the variable and then display it accordingly.

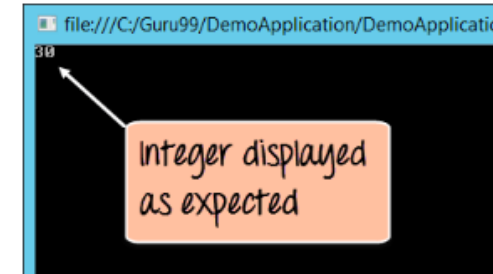


Code Explanation:-

1. The Int32 data type is specified to declare an Integer variable called num. The variable is then assigned a value of 30.
2. Finally the console.write function is used to display the number to the console.

If the above code is entered properly and the program is executed successfully, following output will be displayed.

Output:

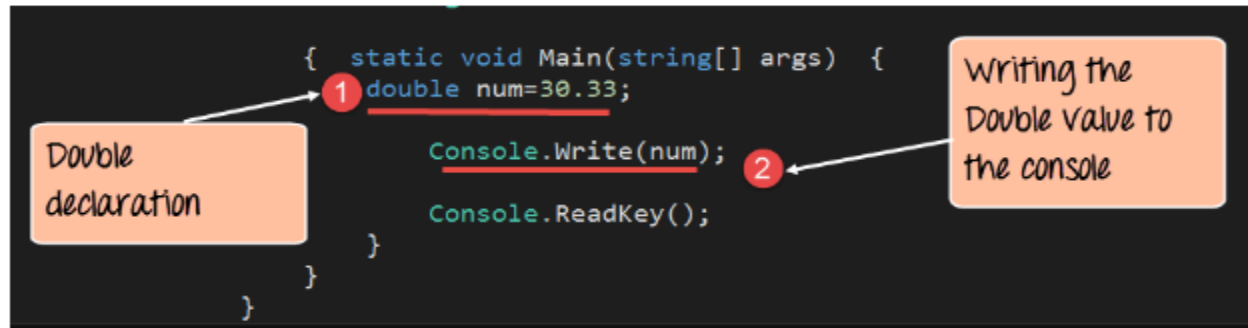


```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
namespace DemoApplication  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            Int32 num=30;  
            Console.Write(num);  
  
            Console.ReadKey();  
        }  
    }  
}
```

2) Double

A double data type is used to work with decimals. In this case, the numbers are whole numbers like 10.11, 20.22 or 30.33. In C#, the datatype is denoted by the keyword “**Double**”. Below is an example of this datatype.

In our example, we will define a double variable called num. We will then assign a Double value to the variable and then display it accordingly.



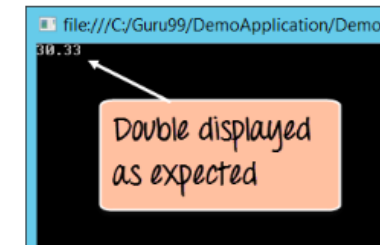
```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
namespace DemoApplication  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            double num=30.33;  
            Console.Write (num) ;  
  
            Console.ReadKey() ;  
        }  
    }  
}
```

Code Explanation:-

1. The double data type is specified to declare a double type **variable** called num. The variable is then assigned a value of 30.33.
2. Finally the console.write function is used to display the number to the console.

If the above code is entered properly and the program is executed successfully, following output will be displayed.

Output:

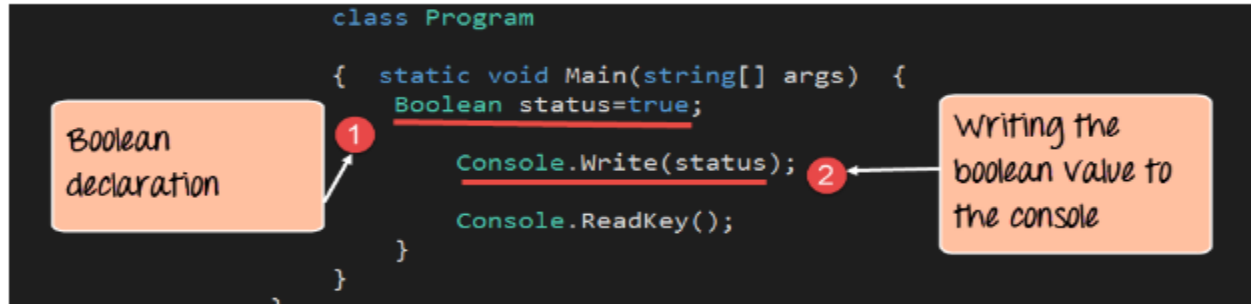


From the output, you can clearly see that the double variable called num was displayed in the console

3) Boolean

A boolean data type is used to work with Boolean values of **true** and **false**. In C#, the datatype is denoted by the Boolean keyword. Below is an example of this datatype can be used.

In our example, we will define a Boolean variable called 'status.' We will then assign a boolean value to the variable and then display it accordingly.



```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace DemoApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            Boolean status=true;
            Console.Write(status);

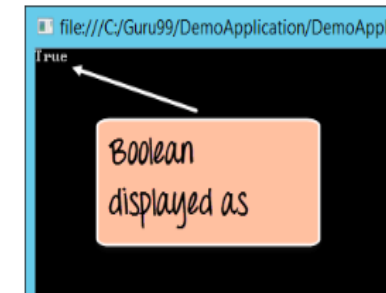
            Console.ReadKey();
        }
    }
}
```

Code Explanation:-

1. The boolean data type is specified to declare a Boolean variable called 'status.' The variable is then assigned a value of true/false.
2. Finally the console.write function is used to display the Boolean value to the console.

If the above code is entered properly and the program is executed successfully, the output will be displayed.

Output:

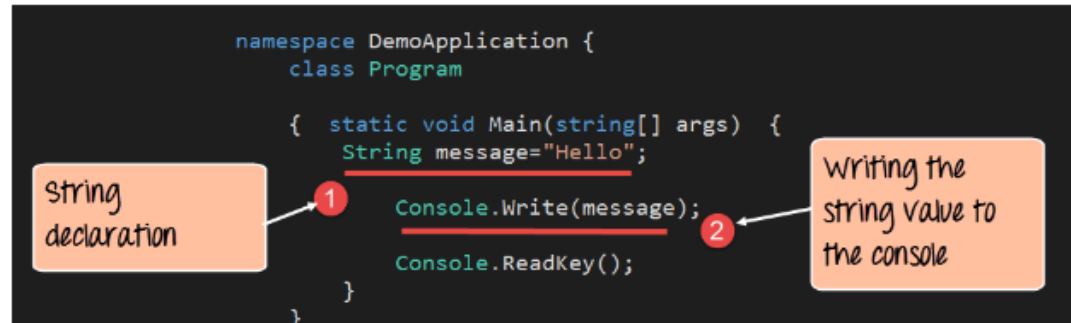


From the output, you can clearly see that the Boolean variable which equals true was displayed in the console

4) String

A String data type is used to work with String values. In C#, the datatype is denoted by the keyword 'String'. Below is an example of this datatype.

In our example, we will define a String variable called 'message.' We will then assign a String value to the variable and then display it accordingly.



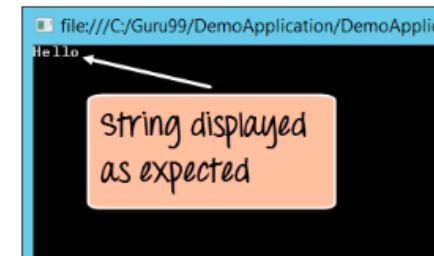
```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
namespace DemoApplication  
{  
    class program  
    {  
        static void Main(string[] args)  
        {  
            String message="Hello";  
            Console.Write(message);  
  
            Console.ReadKey();  
        }  
    }  
}
```

Code Explanation:-

1. The String data type is specified to declare a string variable called message. The variable is then assigned a value of "Hello".
2. Finally, the console.write function is used to display the string value to the console.

If the above code is entered properly and the program is executed successfully, the output will be displayed.

Output:



From the output, you can clearly see that the String variable called message was displayed in the console

AGENDA

- C# Programming
- Module 4 – C# Variables and Operators
- Q&A

C# Variables

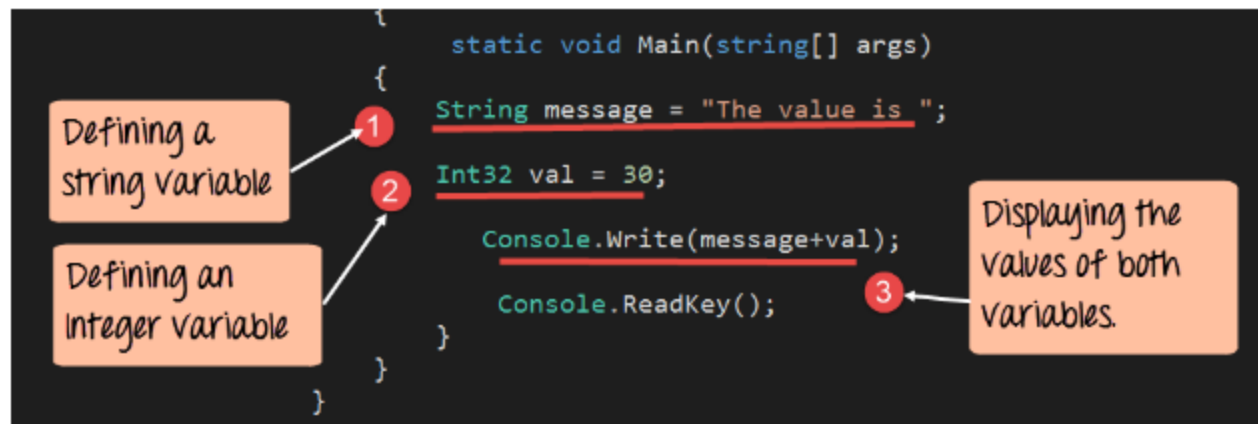
A variable is a name given to a storage area that is used to store values of various data types. Each variable in C# needs to have a specific type, which determines the size and layout of the variable's memory.

For example, a variable can be of the type String, which means that it will be used to store a string value. Based on the data type, specific operations can be carried out on the variable.

For instance, if we had an Integer variable, then operations such as addition and subtraction can be carried out on the variable. One can declare multiple variables in a program.

Let's look at a quick example of the declaration of multiple variables of different data types.

In our example, we will define two variables, one of the type 'string' and the other of the type 'Integer'. We will then display the values of these variables to the console. For each example, we will modify just the main function in our Program.cs file.

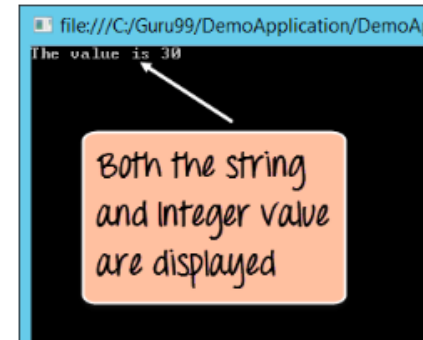


Code Explanation

1. A variable of the data type String is declared. The name of the variable is 'message'. The value of the variable is "The value is ".
2. A variable of the data type Integer (Int32) is declared. The name of the variable is 'val'. The value of the variable is 30.
3. Finally the Console.write statement is used to output both the value of the String and Integer variable.

If the above code is entered properly and the program is executed successfully, the following output will be displayed.

Output



Arithmetic Operators

These are operators used for performing mathematic operations on numbers. Below is the list of operators available in C#.

Operator	Description
+	Adds two operands
-	Subtracts the second operand from the first
*	Multiplies both operands
/	Divides the numerator by de-numerator
%	Modulus Operator and a remainder of after an integer division
++	Increment operator increases integer value by one
--	Decrement operator decreases integer value by one

Relational Operators

These are operators used for performing Relational operations on numbers. Below is the list of relational operators available in C#.

Operator	Description
==	Checks if the values of two operands are equal or not, if yes then condition becomes true.
!=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.

Logical Operators

These are operators used for performing Logical operations on values. Below is the list of operators available in C#.

Operator	Description
&&	This is the Logical AND operator. If both the operands are true, then condition becomes true.
	This is the Logical OR operator. If any of the operands are true, then condition becomes true.
!	This is the Logical NOT operator.

Let's look at a quick example of how the operators can be used in .Net.

In our example, we will define 2 Integer variables and one Boolean variable. We will then perform the following operations

```
namespace DemoApplication {
    class Program
    {
        static void Main(string[] args)
        {
            Int32 val1=10, val2=20;
            bool status = true;
            Console.WriteLine(val1+val2);
            Console.WriteLine(val1 < val2);
            Console.WriteLine(!(status));
            Console.ReadKey();
        }
    }
}
```

The code is annotated with four callouts:

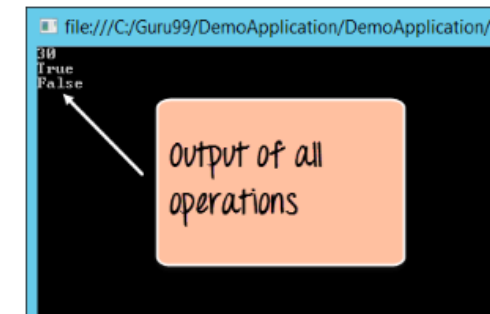
- Defining Variables**: Points to the variable declarations (1).
- Using arithmetic operators**: Points to the addition operation `val1+val2` (2).
- Using Relational operators**: Points to the less-than operation `val1 < val2` (3).
- Using logical operators**: Points to the logical NOT operation `!(status)` (4).

Code Explanation

- Two Integer variables are defined, one being val1 and the other being val2. These will be used to showcase relational and arithmetic operations. A Boolean variable is defined to showcase logical operations.
- An example of the arithmetic operation is shown wherein the addition operator is carried out on val1 and val2. The result is written to the console.
- An example of the relational operation is shown wherein the less than operator is carried out on val1 and val2. The result is written to the console.
- An example of the logical operation is shown, wherein the logical operator (!) is applied to the status variable. The logical NOT operator reverses the current value of any Boolean value. So if a Boolean value is 'true', the logical NOT will return the value 'false' and vice versa. In our case since the value of the status variable is 'true', the result will show 'false'. The result is written to the console.

If the above code is entered properly and the program is executed successfully, the output will be displayed.

Output



AGENDA

- C# Programming
- Module 5 – C# IF, Switch, For, While loop Statement
- Q&A

Flow Control and conditional statements

Flow control and conditional statements are available in any programming language to alter the flow of a program.

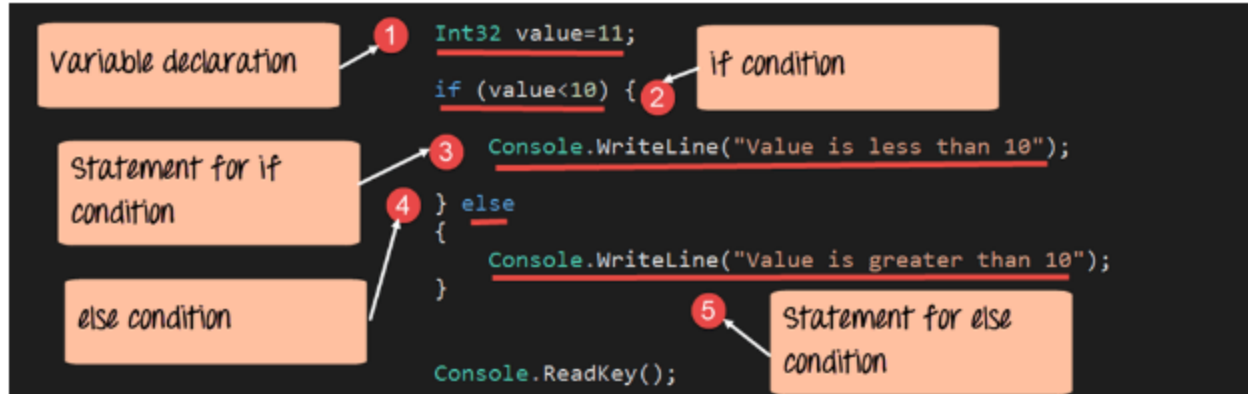
For example, if someone wants to execute only a particular set of statements based on some certain logic, then Flow control, and conditional statements will be useful.

You will get a better understanding as we go through the various statements which are available in C#.

1) If statement

The if statement is used to evaluate a boolean expression before executing a set of statements. If an expression evaluates to true, then it will run one set of statements else it will run another set of statements.

In our example below, a comparison is made for variable called value. If the value of the variable is less than 10, then it will run one statement, or else it will run on another statement.

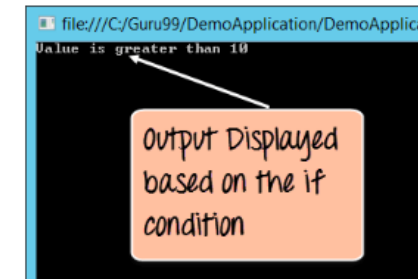


Code Explanation

1. We first define a variable called value and set it to the value of 11.
2. We then use the 'if' statement to check if the value is less than 10 of the variable. The result will either be true or false.
3. If the if condition evaluates to true, we then send the message "Value is less than 10" to the console.
4. If the if condition evaluates to false, we then send the message "Value is greater than 10" to the console.

If the above code is entered properly and the program is executed successfully, the following output will be displayed.

Output:



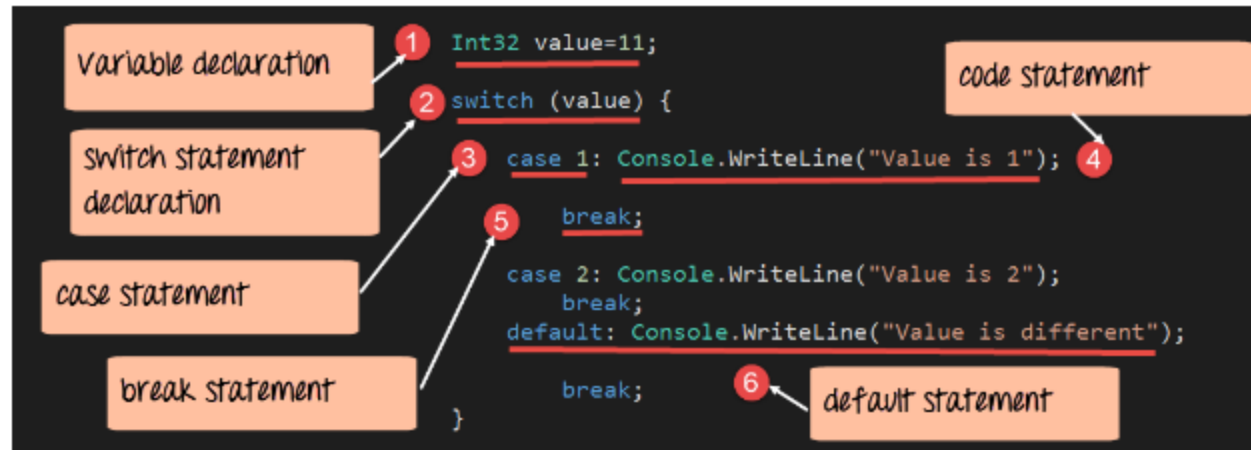
We can clearly see that the 'if' statement was evaluated to false. Hence the message "Value is greater than 10" was sent to the console.

2) Switch statement

The switch statement is an enhancement to the 'if' statement. If you have multiple expressions that need to be evaluated in one shot, then writing multiple 'if' statements becomes an issue.

The switch statement is used to evaluate an expression and run different statements based on the result of the expression. If one condition does not evaluate to true, the switch statement will then move to the next condition and so forth.

Let's see, how this works with the below example. Here, we are again comparing the value of a variable called 'value.' We then check if the value is equal to 1, or 2, or something totally different.

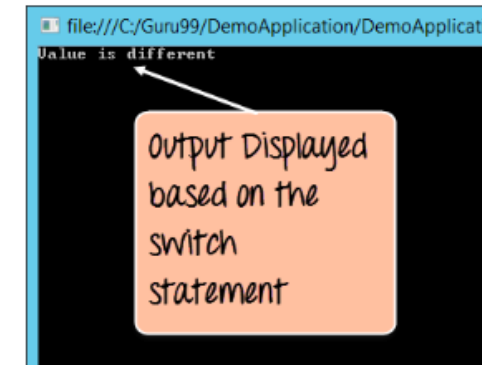


Code Explanation:-

1. We first define a variable called 'value' and set it to the value of 11.
2. We then use the 'switch' statement to check the value of the variable 'value.'
3. Case statements are used to set different conditions. Based on the conditions, a set of statements can be executed. A switch statement can have multiple case conditions. The first case statement checks to see if the value of the variable is equal to 1.
4. If the first case statement is true, then the message "Value is 1" is written to the console.
5. The break statement is used to break from the entire switch statement, once a condition is true.
6. The default condition is a special condition. This just means that if no case expression evaluates to true, then run the set of statements for the default condition.

If the above code is entered properly and the program is executed successfully, the following output will be displayed. The output prints the default value "Value is different", since no condition is satisfied.

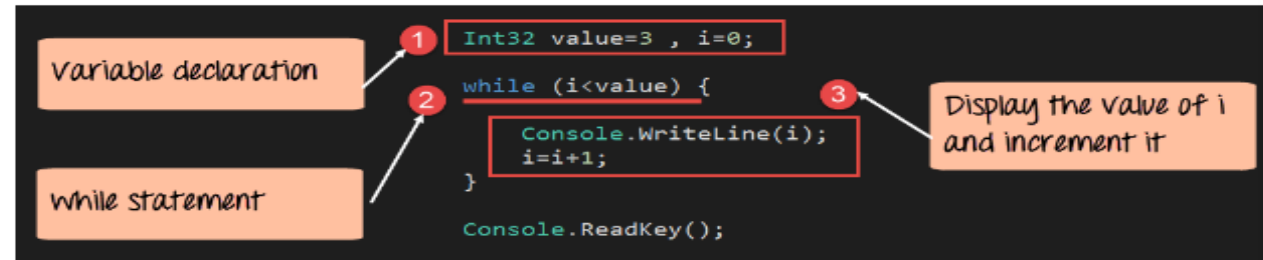
Output:



3) While loop

The while loop is used for iterative purposes. Suppose if you want to repeat a certain set of statements for a particular number of times, then while loop is used.

In our example below, we use the while statement to display the value of a variable 'i'. The while statement is used to display the value 3 times.

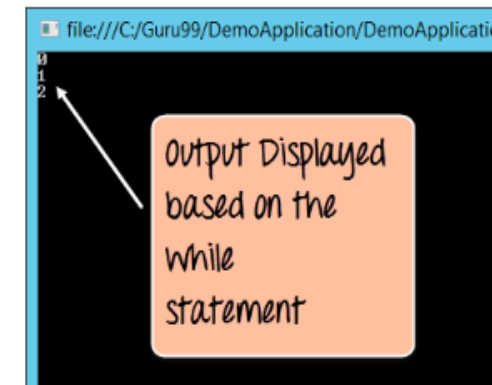


Code Explanation:-

1. Two Integer **variables** are defined, one being value and the other being 'i'. The value variable is used as the upper limit to which we should iterate our while statement. And 'i' is the variable which will be processed during the iteration.
2. In the while statement, the value of 'i' is constantly checked against the upper limit.
3. Here we display the value of 'i' to the console. We also increment the value of 'i'.

If the above code is entered properly and the program is executed successfully, the following output will be displayed.

Output:

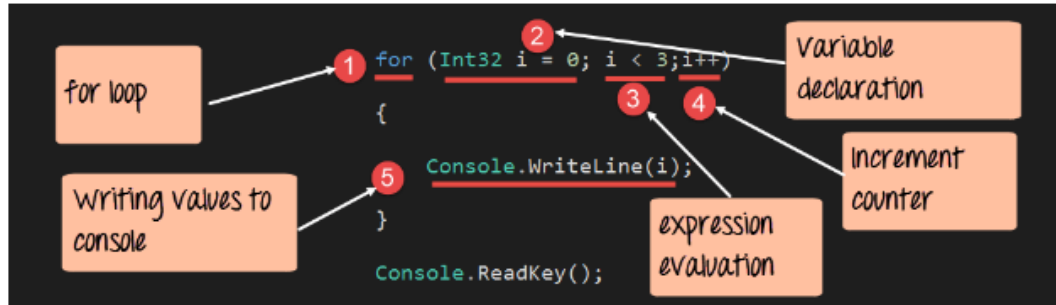


Here you can see that the while statement is executed 3 times and incremented at the same time. And each time, it displayed the current value of the variable 'i'.

4) For loop

The 'for' loop is also used for iterative purposes. Suppose if you want to repeat a certain set of statements for a particular number of times, then forloop is used.

In our example below, we use the 'for' statement to display the value of a variable 'i'. The 'for' statement is used to display the value 3 times.



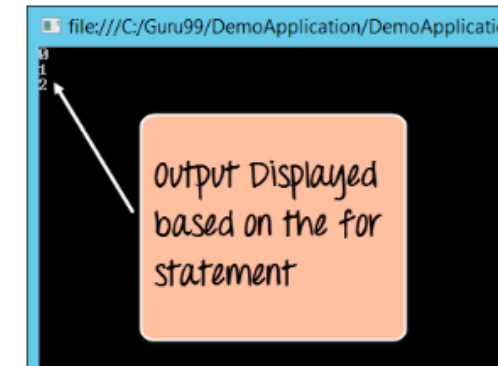
```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
namespace DemoApplication  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            for(Int32 i=0;i<3;i++)  
            {  
                Console.WriteLine(i);  
            }  
            Console.ReadKey();  
        }  
    }  
}
```

Code Explanation:-

1. The 'for' keyword is used to start off the 'for loop' statement.
2. In the 'for loop', we define 3 things. The first is to initialize the value of a variable, which will be used in the 'for loop'.
3. The second is to compare the value of the 'i' against an upper limit. In our case, the upper limit is the value of 3 ($i < 3$).
4. Finally, we increment the value of 'i' accordingly.
5. Here we display the value of 'i' to the console.

If the above code is entered properly and the program is executed successfully, the following output will be displayed.

Output:



Here you can see that the 'for' statement is executed 3 times. And each time, it displayed the current value of the variable 'i'.

AGENDA

- C# Programming
- Module 6 – C# Arrays
- Q&A



What is an Arrays in C#?

An array is used to store a collection or series of elements. These elements will be of the same type.

So for example, if you had an array of Integer values, the array could be a collection of values such as [1, 2, 3, 4]. Here the number of elements in the array is 4.

Arrays are useful when you want to store a collection of values of the same type. So instead of declaring a [variable](#) for every element, you can just declare one variable.

This variable will point to an array or list of elements, which will be responsible for storing the elements of the array.

Let's look at how we can work with arrays in C#. In our example, we will declare an array of Integers and work with them accordingly.

Note that all of the below code is being made to the Program.cs file.

Step 1) Declaring an array – The first step is to declare an array. Let's see how we can achieve this by the below code example.

```
namespace DemoApplication {  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            Int32[] values;  
        }  
    }  
}
```

The diagram highlights the components of the array declaration `Int32[] values;` with numbered callouts and labels:

- 1**: Points to `Int32`, labeled "Data type".
- 2**: Points to `[]`, labeled "shows this is an array".
- 3**: Points to `values`, labeled "Variable name".

Code Explanation:-

1. The first part is the **datatype**. It specifies the type of elements used in the array. So in our case, we are creating an array of Integers.
2. The second part `[]`, which specifies the rank of the array. (The rank is a placeholder which specifies the number of elements the array will contain)
3. Next is the Name of the array which in our case is 'values'. Note you see a green squiggly underline, don't worry about that. That is just .Net saying that you have declared an array, but not using it anywhere.

Step 2) The next step is to initialize the array. Here we are going to specify the number of values the array will hold. We are also going to assign values to each element of the array.

```
static void Main(string[] args)
{
    Int32[] values;

    values = new Int32[3]; 1
    values[0] = 1;
    values[1] = 2;
    values[2] = 3; 2
}
```

Assigning values to the elements of the array

Array is going to hold 3 elements

Code Explanation:-

1. First, we are setting the number of elements the array will hold to 3. So in the square brackets, we are saying that the array will hold 3 elements.
2. Then we are assigning values to each element of the array. We can do this by specifying the variable name + the index position in the array. So values[0] means that we are storing a value in the first position of the array. Similarly to access the second position, we use the notation of values[1] and so on and so forth.

Note: – In Arrays, the index position starts from 0.

Step 3) Let's now display the individual elements of the array in the Console. Let's add the below code to achieve this.

```
values = new Int32[3];

values[0] = 1;
values[1] = 2;
values[2] = 3;

Console.WriteLine(values[0]);
Console.WriteLine(values[1]);
Console.WriteLine(values[2]);

Console.ReadKey();
```

using
Console.WriteLine
to send each
element value to
the console

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace DemoApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            Int32[] value;
            value=new Int32[3];

            value[0]=1;
            value[1]=2;
            value[2]=3;

            Console.WriteLine(value[0]);
            Console.WriteLine(value[1]);
            Console.WriteLine(value[2]);

            Console.ReadKey();
        }
    }
}
```

Code Explanation:-

This is the simple part wherein we just use the Console.WriteLine method to send each value of the element to the console.

Note that again, we are accessing each element with the help of the array variable name along with the index position.

If the above code is entered properly and the program is executed, the following output will be displayed.

Output:

```
file:///C:/Guru99/DemoApplication/DemoApplicatio
1
2
3
```

output of all the
values of the
array

From the output, you can see all the values of the array being displayed in the Console.

AGENDA

- C# Programming
- Module 7 – C# Class and Object
- Q&A

C# is based on the C++ programming language. Hence, the C# programming language has in-built support for classes and objects. A class is nothing but an encapsulation of properties and methods that are used to represent a real-time entity.

For example, if you want to work with employee's data in a particular application.

The properties of the employee would be the ID and name of the employee. The methods would include the entry and modification of employee data.

The name of the class is "Tutorial". The class has the following properties

- **Tutorial ID** – This will be used to store a unique number which would represent the Tutorial.
- **Tutorial Name** – This will be used to store the name of the tutorial as a string.

A class also comprises of methods. Our class has the following methods,

- **SetTutorial** – This method would be used to set the ID and name of the Tutorial. So for example, if we wanted to create a tutorial for .Net, we might create an object for this. The object would have an ID of let's say 1. Secondly, we would assign a name of ".Net" as the name of the Tutorial. The ID value of 1 and the name of ".Net" would be stored as a property of the object.
- **GetTutorial** – This method would be used to get the details of a specific tutorial. So if we wanted to get the name of the Tutorial, this method would return the string ".Net".

Tutorial Class

- Properties
 - TutorialID
 - TutorialName
- Methods
 - SetTutorial
 - GetTutorial

Below is a snapshot of how an object might look like for our Tutorial class. We have 3 objects, each with their own respective TutorialID and TutorialName.

TutorialObject1

- TutorialID =1
- TutorialName = C#

TutorialObject2

- TutorialID =2
- TutorialName = ASP.Net

TutorialObject3

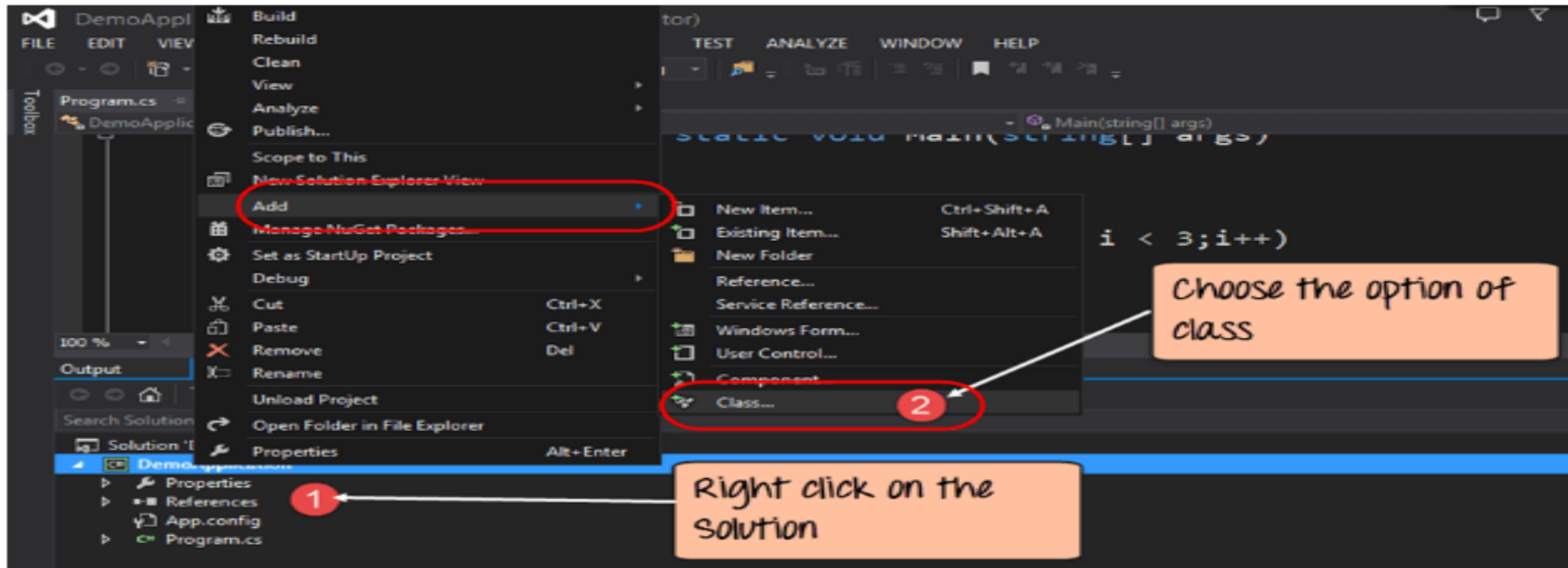
- TutorialID =3
- TutorialName = VB.Net

How to Create a Class and Object

Let's now dive into **Visual Studio** to create our class. We are going to build upon our existing console application which was created in our earlier chapter. We will create a class in Visual Studio for our current application.

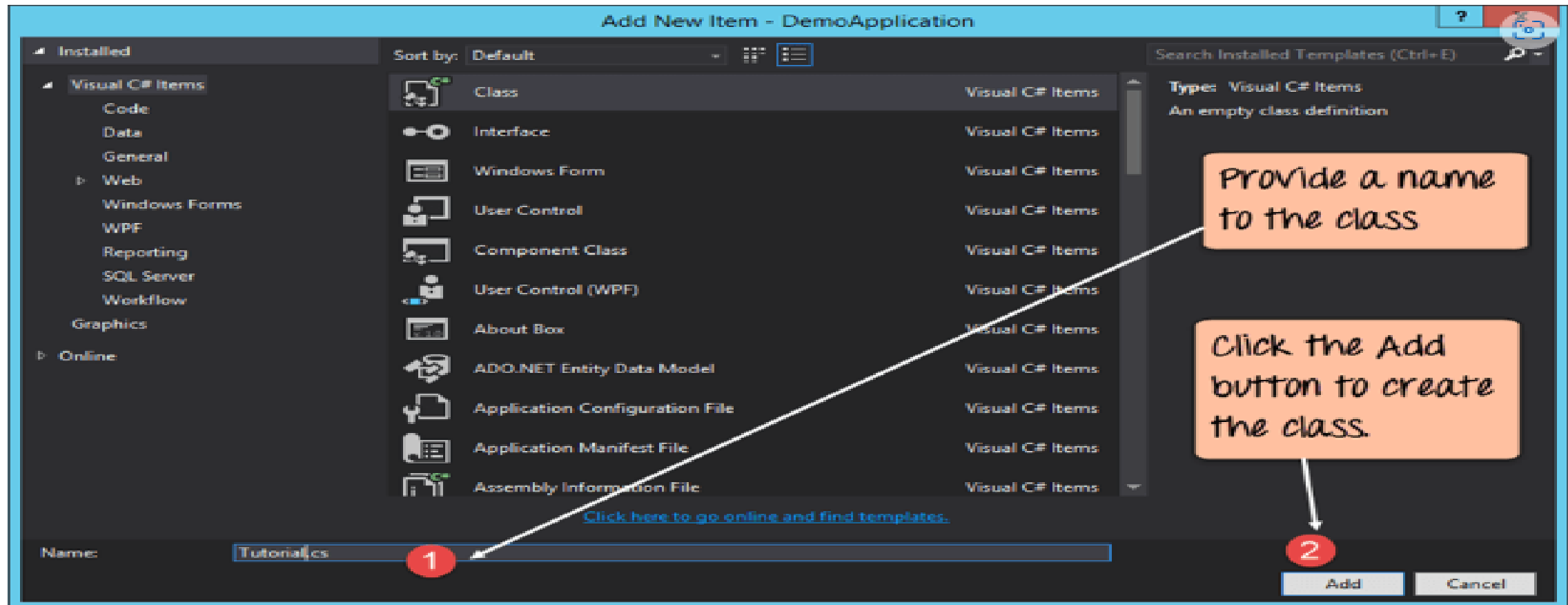
Let's follow the below-mentioned steps to get this example in place.

Step 1) The first step involves the creation of a new class within our existing application. This is done with the help of Visual Studio.



1. The first step is to the right click on the solution, which in our case is 'DemoApplication'. This will bring up a context menu with a list of options.
2. From the context menu choose the option Add->Class. This will provide the option to add a class to the existing project.

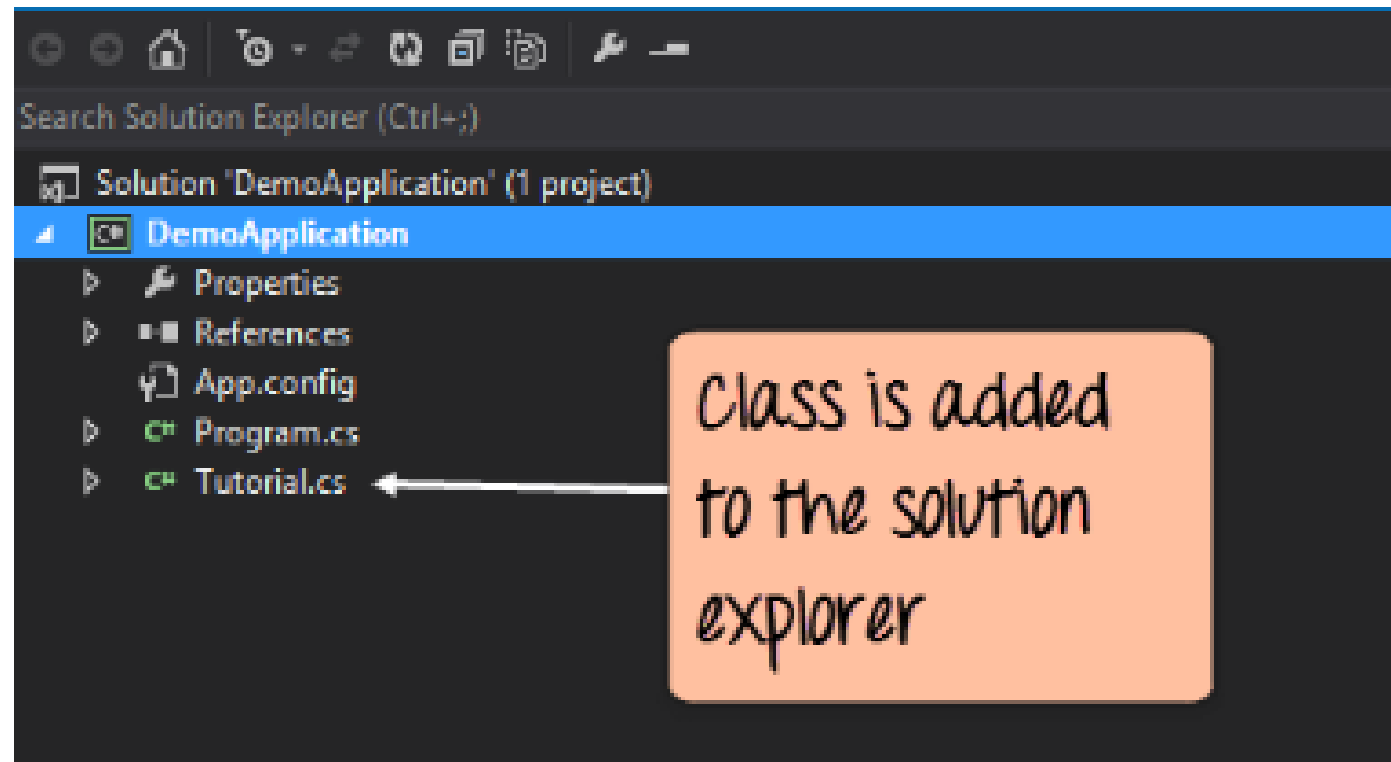
Step 2) The next step is to provide a name for the class and add it to our solution.



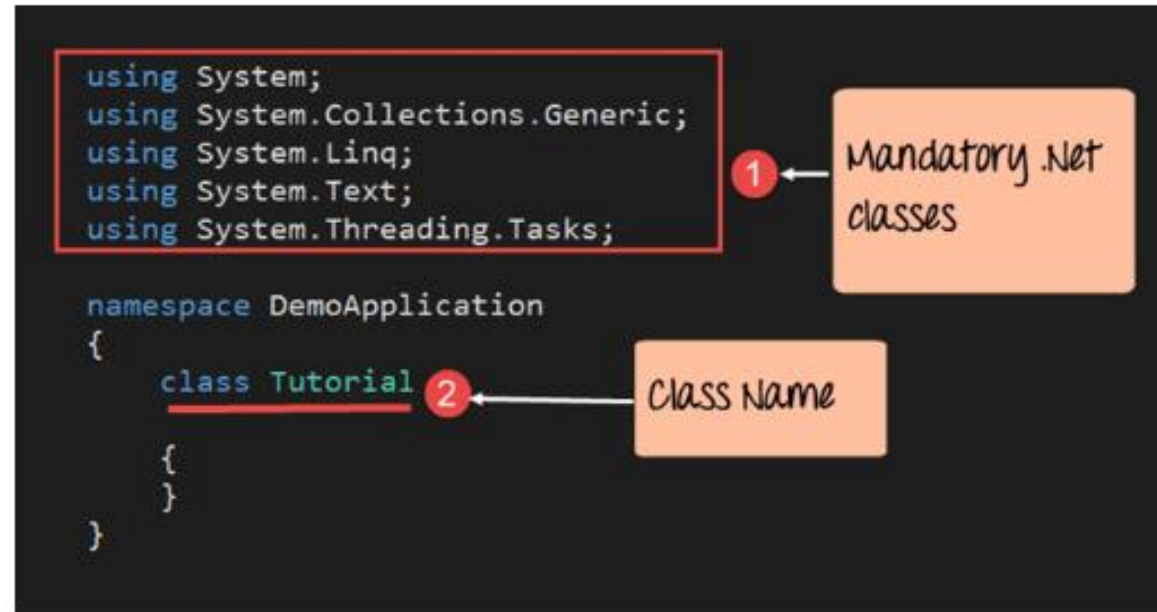
1. In the project dialog box, we first need to provide a name for our class. Let's provide a name of Tutorial.cs for our class. Note that the file name should end with .cs to ensure it is treated as a proper class file.
2. When we click the Add button, the class will be added to our solution.

If the above steps are followed, you will get the below output in Visual Studio.

Output:-



A class named Tutorial.cs will be added to the solution. If you open the file, you will find the below code added to the class file.



Code Explanation:-

1. The first part contains the mandatory modules which Visual Studio adds to any .Net file. These modules are always required to ensure any .Net program runs in a Windows environment.
2. The second part is the class which is added to the file. The class name is 'Tutorial' in our case. This is the name which was specified with the class was added to the solution.

For the moment, our class file does not do anything. In the following topics, we will look into more details on how to work with the class.

Fields and methods

We have already seen how fields and methods are defined in classes in the earlier topic.

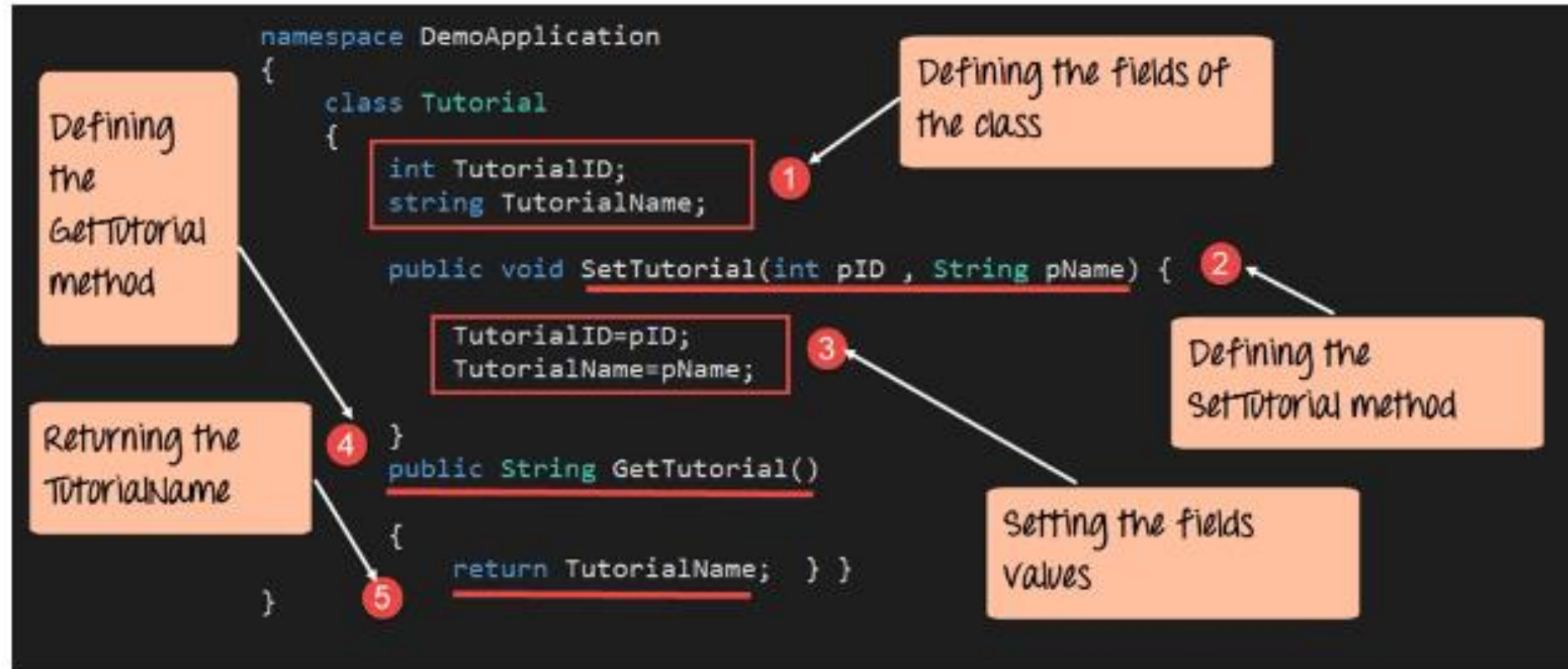
For our Tutorial class, we can have the following properties.

1. Tutorial ID – This will be used to store a unique number which would represent the Tutorial.
2. Tutorial Name – This will be used to store the name of the tutorial as a [string](#).

Our Tutorial class can also have the below-mentioned methods.

1. SetTutorial – This method would be used to set the ID and name of the Tutorial.
2. GetTutorial – This method would be used to get the details of a specific tutorial.

Step 1) The first step is to ensure the Tutorial class has the right fields and methods defined. In this step, we add the below code to the Tutorial.cs file.



Code Explanation:-

1. The first step is to add the fields of TutorialID and TutorialName to the class file. Since the TutorialID field will be a number, we define it as an Integer, while TutorialName will be defined as a string.
2. Next, we define the SetTutorial method. This method accepts 2 parameters. So if Program.cs calls the SetTutorial method, it would need to provide the values to these parameters. These values will be used to set the fields of the Tutorial object.

Note: let's take an example and assume our Program.cs file calls the SetTutorial with the parameters "1" and ".Net". The below steps would be executed as a result of this,

1. The value of pID would become 1
 2. The value of pName would be .Net.
 3. In the SetTutorial method, these values would then be passed to TutorialID and TutorialName.
 4. So now TutorialID would have a value of 1 and TutorialName would have a value of ".Net".
-
3. Here we set the fields of the Tutorial class to the parameters accordingly. So we set TutorialID to pID and TutorialName to Pname.
 4. We then define the GetTutorial method to return value of the type "String". This method will be used to return the TutorialName to the calling program. Likewise, you can also get the tutorial id with method Int GetTutorial
 5. Here we return the value of the TutorialName field to the calling program.

Step 2) Now let's add code to our Program.cs, which is our Console application. The Console application will be used to create an object of the "Tutorial class" and call the SetTutorial and GetTutorial methods accordingly.

(**Note:-** An object is an instance of a class at any given time. The difference between a class and an object is that the object contains values for the properties.)

```
static void Main(string[] args)
{
    Tutorial pTutor = new Tutorial(); 1
    pTutor.SetTutorial(1, ".Net"); 2
    Console.WriteLine(pTutor.GetTutorial());
    Console.ReadKey();
}
```

The diagram illustrates the execution of the provided C# code. It features three callout boxes with arrows pointing to specific lines of code:

- Creating a new Tutorial object:** Points to line 1, `Tutorial pTutor = new Tutorial();`.
- Calling the SetTutorial method:** Points to line 2, `pTutor.SetTutorial(1, ".Net");`.
- Calling the GetTutorial method:** Points to line 3, `Console.WriteLine(pTutor.GetTutorial());`.

AGENDA

- C# Programming
- Module 8 – C# Access Modifiers
- Q&A

What is Access Modifier (Specifier) in C#?

Access Modifiers or Access Specifiers in C# are the keywords used to define the visibility of a class property or method. It is used when you don't want other programs to see the properties or methods of a class.

Access modifiers restrict access so that other programs cannot see the properties or methods of a class.

There are 6 types of access modifiers in C#:

- Private
- Public
- Protected
- Internal
- Protected Internal
- Private Protected

Private Access Modifiers in C#

When Private access modifier is attached to either a property or a method, it means that those members cannot be accessed from any external program.

Example of Private Access Modifier

Let's take an example and see what happens when we use the private access modifier.

Let's modify the current code in our Tutorial.cs file. In the SetTutorial method, let's change the public keyword to private.

```
namespace DemoApplication
{
    class Tutorial
    {
        int TutorialID;
        string TutorialName;

        private void SetTutorial(int pID , String pName) {
            TutorialID=pID;
            TutorialName=pName;
        }
    }
}
```

Added the private keyword

C# Public Access Modifiers

When Public access modifier is attached to either a property or a method, it means that those members can be accessed from any external program. We have already seen this in our earlier examples.



```
int TutorialID;  
string TutorialName;  
  
public void SetTutorial(int pID , String pName) {  
    TutorialID=pID;  
    TutorialName=pName;  
}  
  
public String GetTutorial()  
{
```

Since we have defined our methods as public in the Tutorial class, they can be accessed from the Program.cs file.

Protected Access Modifiers in C#

When Protected access modifier is attached to either a property or a method, it means that those members can be accessed only by [classes inherited](#) from the current [class](#). This will be explained in more detail in the Inheritance class.

C# Internal Access Modifiers

When an Internal access modifier is attached to either a property or a method, those members can be accessed only by an internal program but not by an external program.

C# Constructor

[C#](#) Constructors are used to initializing the values of class fields when their corresponding objects are created. A constructor is a method which has the same name as that of the class. If a constructor is defined in a class, then it will provide the first method which is called when an object is created. Suppose if we had a class called Employee. The constructor method would also be named as Employee().

The following key things need to be noted about constructor methods

- 1.The C# default access modifier for the constructor needs to be made as public.
- 2.There should be no return type for the constructor method.

AGENDA

- C# Programming
- Module 9 – C# Array List
- Q&A



What is ArrayList in C#?

The ArrayList collection is similar to the Arrays data type in C#. The biggest difference is the dynamic nature of the array list collection. For arrays, you need to define the number of elements that the array can hold at the time of array declaration. But in the case of the Array List collection, this does not need to be done beforehand. Elements can be added or removed from the Array List collection at any point in time. Let's look at the operations available for the array list collection in more detail.

Declaration of an Array List

The declaration of an ArrayList is provided below. An array list is created with the help of the ArrayList Datatype. The “new” keyword is used to create an object of an ArrayList. The object is then assigned to the variable a1. So now the variable a1 will be used to access the different elements of the array list.

```
ArrayList a1 = new ArrayList()
```

Adding elements to an array

The add method is used to add an element to the ArrayList. The add method can be used to add any sort of data type element to the array list. So you can add an Integer, or a string, or even a Boolean value to the array list. The general syntax of the addition method is given below

```
ArrayList.add(element)
```

Below are some examples of how the “add” method can be used. The add method can be used to add various data types to the Array List collection.

Below you can see examples of how we can add Integer’s Strings and even Boolean values to the Array List collection.

- `a1.add(1)` – This will add an Integer value to the collection
- `a1.add(“Example”)` – This will add a String value to the collection
- `a1.add(true)` – This will add a Boolean value to the collection

Now let’s see this working at a code level. All of the below-mentioned code will be written to our Console application. The code will be written to our Program.cs file.

In the program below, we will write the code to create a new array list. We will also show to add elements and to display the elements of the Array list.

```
static void Main(string[] args)
{
```

```
    ArrayList a1 = new ArrayList();
```

1

Defining an array list

```
    a1.Add(1);
    a1.Add("Example");
    a1.Add(true);
```

2

Adding elements
to the array list

```
    Console.WriteLine(a1[0]);
    Console.WriteLine(a1[1]);
    Console.WriteLine(a1[2]);
```

3

Displaying the elements
of the array list

```
    Console.ReadKey();
}
```

```
1
Example
True
```

The elements of the
array list displayed in
the output

Count

This method is used to get the number of items in the ArrayList collection. Below is the general syntax of this statement.
ArrayList.Count() – This method will return the number of elements that the array list contains.

Contains

This method is used to see if an element is present in the ArrayList collection. Below is the general syntax of this statement
ArrayList.Contains(element) – This method will return true if the element is present in the list, else it will return false.

RemoveAt

This method is used to remove an element at a specific position in the ArrayList collection. Below is the general syntax of this statement
ArrayList.RemoveAt(index) – This method will remove an element from a specific position of the Array List.
Now let's see this working at a code level. All of the below-mentioned code will be written to our Console application.
The code will be written to our Program.cs file.

Removing an element and showing that the element has been removed

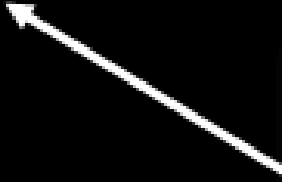
```
{  
    static void Main(string[] args)  
    {  
        ArrayList a1 = new ArrayList();  
  
        a1.Add(1);  
        a1.Add("Example");  
        a1.Add(true);  
  
        Console.WriteLine(a1.Count);  
  
        Console.WriteLine(a1.Contains(2));  
  
        Console.WriteLine(a1[1]);  
        a1.RemoveAt(1);  
        Console.WriteLine(a1[1]);  
    }  
}
```

Count of items in the Array list

Checking to see if the Array List contains the element

3

```
3  
False  
Example  
True
```



Output of the various
Array List functions